Projet Python — Moteur d'aventure textuelle

Bachelor 1 — Programmation Python — Seul ou en binôme — environ 50 heures

Principe

Vous développez un **moteur de jeu d'aventure textuelle**, dans l'esprit des premiers jeux d'aventure (Colossal Cave Adventure, Zork). Le joueur tape des commandes en français, comme « prendre la clé en bronze » ou « aller au nord » ; le moteur les comprend et fait évoluer le monde : déplacements, objets, portes fermées, salles plongées dans le noir, coffres à code.

Point essentiel : **le contenu du jeu n'est pas écrit dans le code**. Les salles, les objets et les énigmes sont lus dans un fichier JSON, le **scénario**. Le même moteur doit faire tourner n'importe quel scénario qui respecte le format décrit en annexe : celui qui est fourni, et celui que vous écrirez.

Fourni : `scenarios/manoir.json`, « Le Manoir de Python » : 9 salles, 9 objets, 2 portes fermées à clé, un passage bloqué, une salle sombre et un coffre à code. Le format complet est décrit dans l'annexe A, les commandes attendues dans l'annexe B.

Le projet se fait **seul ou en binôme**, pour environ **50 heures** de travail sur 6 semaines, dont une large part à la maison.

Partie commune (obligatoire)

A. Le scénario

- Le joueur choisit un scénario parmi les fichiers `.json` du dossier `scenarios/` (module `pathlib`).
- Le moteur **valide** le scénario avant de le lancer : champs obligatoires présents, salles de départ et d'arrivée existantes, chaque sortie mène à une salle existante, chaque objet cité existe. Il affiche **toutes** les erreurs trouvées, pas seulement la première, et refuse de lancer un scénario invalide. → **Terminé quand** : un scénario avec trois erreurs volontaires affiche les trois.

B. L'analyseur de commandes

- Insensible aux majuscules, aux accents et aux espaces en trop ; les petits mots sont ignorés (le, la, l', les, un, du, vers…).
- Plusieurs verbes pour une même action (prendre, ramasser ; examiner, lire…), directions abrégées (n, s, e, o) ou seules (« nord » = « aller nord »).
- Un objet peut être désigné par plusieurs mots (« la clé en bronze ») ou par un synonyme défini dans le scénario (« clef »). Si la désignation est ambiguë, le moteur demande laquelle : « Laquelle ? clé en bronze, clé en argent ».
- Une commande inconnue donne un message utile, jamais une erreur Python. → **Terminé quand** : « Prendre la CLEF » fonctionne comme « prendre clé ».

C. Le monde

- Se déplacer, regarder, examiner, prendre, poser, afficher l'inventaire.
- Chaque objet a un **poids** ; le joueur a une **capacité** maximale. Certains objets sont fixes et ne peuvent pas être pris.
- Des **sorties bloquées** (porte à clé, passage obstrué) s'ouvrent avec « utiliser <objet> ».
- Des **salles sombres** : sans objet lumineux dans l'inventaire, on n'y voit ni la description ni les objets.
- Des **objets à code** (un coffre) : « ouvrir coffre » demande un code ; s'il est juste, le contenu apparaît dans la salle.

D. La partie

- Un compteur de tours (chaque action qui modifie le monde compte). La partie est gagnée en atteignant la salle d'arrivée.
- **Sauvegarde et chargement** dans 3 emplacements, en JSON. On enregistre seulement l'**état** de la partie (salle, inventaire, objets déplacés, sorties ouvertes, coffres ouverts, tours), et on refuse de charger la sauvegarde d'un autre scénario.
- Un **classement** des meilleures parties par scénario (le moins de tours possible), dans `scores.json`.

E. Architecture et qualité

- **Programmation objet obligatoire** : au minimum les classes `Objet`, `Salle`, `Joueur` et `Jeu`. L'analyseur est dans un module séparé. Aucun texte du scénario n'apparaît dans le code.
- La méthode `executer(commande)` de `Jeu` reçoit une commande et renvoie le texte à afficher : c'est ce qui rend le moteur testable.
- Une docstring sur chaque fonction et chaque classe ; aucun plantage, quelle que soit la saisie ou l'état des fichiers.

F. Tests automatiques

- Un fichier `tests.py` avec au moins **15 assertions** : analyseur, validation de scénarios invalides, sauvegarde puis chargement, et une **partie complète du manoir jouée automatiquement** à partir d'une liste de commandes, jusqu'à la victoire.

G. Votre scénario

- Écrivez votre propre scénario : au moins **10 salles**, **12 objets**, **3 obstacles ou énigmes**, une salle sombre et un objet à code. Il doit être terminable : sa solution complète figure dans `tests.py`.

Modules au choix

Niveau 1

- **Confort de jeu** : « encore » répète la dernière commande, « historique » affiche les 10 dernières, « aide prendre » explique un verbe.
- **Objets combinables** : « combiner corde crochet » crée un grappin ; les combinaisons possibles sont décrites dans le JSON.
- **Score et rangs** : des points pour chaque objet trouvé ou énigme résolue, définis dans le JSON, et un rang final (« Aventurier débutant », « Explorateur »…).

Niveau 2

- **Personnages et dialogues** : des personnages avec un arbre de dialogue en JSON (réponses numérotées), et des conditions : donner un objet pour obtenir une information ou un passage.
- **Carte ASCII** : affichage, en grille, des salles déjà visitées et de leurs liaisons, positionnées à partir des directions.
- **Éditeur de scénario** : créer et modifier salles, objets et sorties avec des menus, en validant le scénario avant de l'enregistrer.
- **Événements** : des événements déclenchés par le nombre de tours ou par une action (la lampe s'éteint après 40 tours, une porte se referme derrière vous…), décrits dans le JSON.

Niveau 3

- **Combats** : des ennemis avec des points de vie, des armes avec des dégâts, des combats au tour par tour avec une part de hasard ; en cas de défaite, reprise de la dernière sauvegarde.
- **Solveur automatique** : un programme qui trouve seul la suite de commandes la plus courte pour gagner un scénario, par un parcours en largeur des états du jeu (**non vu en cours**). Il prouve aussi qu'un scénario est terminable.
- **Générateur de donjon** : création d'un scénario aléatoire valide, où aucune clé n'est enfermée derrière la porte qu'elle ouvre ; vérification par le solveur ou par un parcours des salles.

Exigence minimale

- **Seul** : partie commune + **3 modules**, dont au moins 1 de niveau 2.
- **Binôme** : partie commune + **5 modules**, dont au moins 2 de niveau 2 et 1 de niveau 3.

Bonus (points supplémentaires)

Chaque bonus vaut 1 point, dans la limite de 20. Ils ne sont pas nécessaires pour avoir 20.

- Un affichage en couleurs dans le terminal (codes ANSI, ou la bibliothèque `rich` installée avec pip).
- Un moteur bilingue : tous les messages du moteur dans un fichier de langue JSON, en français et en anglais.
- Un scénario supplémentaire qui utilise tous vos modules.
- **Non vu en cours** : une interface graphique avec `tkinter`, ou un analyseur écrit avec les expressions régulières (module `re`).

Organisation conseillée (environ 50 heures)

Jalon	Contenu	Durée
1	Classes, chargement du manoir, déplacements, « regarder »	8 h
2	Analyseur complet, objets, inventaire, poids	10 h
3	Obstacles, salles sombres, coffres, victoire, validation des scénarios	8 h
4	Sauvegardes, classement, tests automatiques	6 h
5	Modules choisis	12 h
6	Votre scénario, robustesse, README, préparation de la soutenance	6 h

À chaque jalon, vous montrez votre avancement au formateur (5 minutes en séance) : ce suivi compte dans la note. Un dépôt Git avec des commits réguliers est fortement conseillé ; en binôme, chacun doit y apparaître.

Ce que vous devez rendre

- Le dossier du projet (archive zip ou dépôt Git) : le code, le dossier `scenarios/` avec le manoir et votre scénario, `tests.py`.
- Un fichier `README.md` : comment lancer le jeu et les tests, les modules réalisés, la solution de votre scénario, et en binôme la répartition du travail.

Barème (sur 20)

Critère	Points
Moteur : chargement, validation, monde, obstacles, victoire	5
Analyseur de commandes	2
Sauvegardes et classement	2
Architecture objet et qualité du code	2
Tests automatiques	1
Votre scénario	1
Modules choisis	3
Suivi des jalons	1
Soutenance	3
Total	**20**

Soutenance (20 minutes)

1. **3 minutes** — Le manoir joué jusqu'à la victoire.
2. **5 minutes** — Votre scénario et vos modules.
3. **7 minutes** — Modification du code en direct, demandée par le formateur (par exemple : ajouter un verbe, un nouveau type d'obstacle). En binôme, chacun modifie une partie qu'il n'a pas écrite.
4. **5 minutes** — Questions sur le code et les notions du cours.

Chaque membre doit pouvoir expliquer **l'intégralité** du code rendu. Les notions hors cours et les assistants d'IA sont autorisés pour apprendre et comprendre, à condition de savoir expliquer chaque ligne : un code que vous ne savez pas expliquer ne rapporte pas de points.

Annexe A — Le format du scénario

Le fichier

Champ	Rôle
`titre`, `intro`	Le nom du scénario et le texte affiché au lancement.
`depart`, `arrivee`	Les identifiants de la salle de départ et de la salle à atteindre pour gagner.
`victoire`	Le texte affiché quand le joueur gagne.
`capacite`	Le poids maximal que le joueur peut porter (6 par défaut).
`salles`	Un dictionnaire : identifiant de salle → description de la salle.
`objets`	Un dictionnaire : identifiant d'objet → description de l'objet.
`synonymes`	Facultatif : mot → liste de synonymes, par exemple `"cle": ["clef"]`.

Une salle

Champ	Rôle
`nom`, `description`	Le titre affiché et le texte de la salle.
`sorties`	Direction (nord, sud, est, ouest, haut, bas) → identifiant de la salle d'arrivée, ou une sortie bloquée (ci-dessous).
`objets`	La liste des identifiants des objets présents au départ.
`sombre`	Facultatif : `true` si la salle est invisible sans objet lumineux.

Une sortie bloquée

Champ	Rôle
`vers`	L'identifiant de la salle d'arrivée.
`requiert`	L'identifiant de l'objet qui ouvre le passage avec « utiliser ».
`bloquee`, `ouverture`	Le texte affiché tant que le passage est fermé, et au moment où il s'ouvre.

Un objet

Champ	Rôle
`nom`, `description`	Le nom affiché (« clé en bronze ») et le texte donné par « examiner ».
`poids`	Le poids de l'objet (0 par défaut).
`fixe`	Facultatif : `true` si l'objet ne peut pas être pris.
`lumiere`	Facultatif : `true` si l'objet éclaire les salles sombres.
`code`, `contient`	Facultatif : le code qui ouvre l'objet, et la liste des objets qui apparaissent alors.

Exemple de sortie bloquée : `"ouest": {"vers": "bibliotheque", "requiert": "cle_bronze", "bloquee": "La porte est fermée à clé.", "ouverture": "La porte s'ouvre."}`. Vous pouvez ajouter des champs pour vos modules : documentez-les dans votre README.

Annexe B — Les commandes à reconnaître

Commande	Effet
`aller <direction>`, `n`, `s`, `e`, `o`, `haut`, `bas`	Se déplacer.
`regarder`	Décrire la salle, ses objets visibles et ses sorties.
`examiner <objet>`	Afficher la description d'un objet de la salle ou de l'inventaire.
`prendre <objet>`, `poser <objet>`	Ramasser un objet (si le poids le permet) ou le déposer.
`utiliser <objet>`	Ouvrir les sorties de la salle qui requièrent cet objet.
`ouvrir <objet>`	Ouvrir un objet à code : le moteur demande le code.
`inventaire`	Lister les objets portés et le poids total.
`sauver`, `charger`, `aide`, `quitter`	Gérer la partie.